
Redes Neuronales Artificiales
Examen Parcial 2:
Optimización de MLP y SOM mediante
Algoritmos Genéticos

Jairo Isaac Muñoz López
2020170203

Universidad Tecnológica de la Mixteca
Huajuapán de León, Oaxaca, México.

Índice general

0.1. Introducción	4
0.2. Descripción del Algoritmo de Optimización	4
0.3. Parte A: Perceptrón Multicapa (MLP)	4
0.3.1. Implementación y Optimización	4
0.3.2. Resultados MLP	5
0.3.3. Análisis de Resultados (MLP)	5
0.4. Comparación con una configuración distinta	6
0.5. Parte B: Mapa Autoorganizado (SOM)	6
0.5.1. Implementación Base	6
0.5.2. Resultados de Optimización SOM	7
0.5.3. Análisis de Resultados (SOM)	7
0.6. Análisis de Resultados (SOM)	7
0.7. Comparación con otra configuración	8
0.8. Conclusión	8
0.9. Ejecución	8

0.1. Introducción

El presente reporte detalla la implementación y optimización de dos modelos de redes neuronales: el Perceptrón Multicapa (MLP) para clasificación supervisada y el Mapa Autoorganizado (SOM) para aprendizaje no supervisado. Se utiliza el **Wine Dataset**, el cual contiene 178 muestras de 13 características fisicoquímicas de tres tipos de vino.

El motivo por el que se desidió utilizar este enfoque radica en la complejidad del espacio de hiperparámetros, pues en prácticas previas pudimos notar que la optimización manual es poco eficiente. El uso de Algoritmos Genéticos (GA) mediante la biblioteca `pyGAD` permite realizar una búsqueda heurística eficiente para encontrar arquitecturas de red y configuraciones de entrenamiento que maximicen la precisión (*Accuracy*) en el MLP y el Índice de Rand Ajustado (ARI) en el SOM. Esto nos evita el problema de ajustar los parámetros de manera intuitiva.

0.2. Descripción del Algoritmo de Optimización

Para ambas partes se empleó un Algoritmo Genético con las siguientes características:

- **Representación del Cromosoma:** Un vector de valores reales y enteros que codifican parámetros como tasa de aprendizaje, dimensiones del mapa, funciones de activación, número de iteraciones, número de neuronas, etc.
- **Función de Aptitud (Fitness):**
 - Parte A: Exactitud (*Accuracy*) del MLP sobre el conjunto de prueba. Nos permite evaluar la capacidad del modelo para clasificar correctamente las muestras de vino. Entre más alto sea el valor, mejor será la configuración de hiperparámetros.
 - Parte B: *Adjusted Rand Index* (ARI) comparando los clusters del SOM + KMeans contra las etiquetas reales. El ARI mide la similitud entre dos agrupamientos, ajustando por el azar. Un valor de ARI cercano a 1 indica una excelente correspondencia entre los clusters generados por el SOM y las clases reales del dataset.
- **Operadores Genéticos:** Cruce de un punto con probabilidad de 0.9 y mutación aleatoria con probabilidad de 0.1.
- **Criterio de Paro:** Ejecución por un número fijo de 50 generaciones.

0.3. Parte A: Perceptrón Multicapa (MLP)

0.3.1. Implementación y Optimización

Se utilizó `MLPClassifier` de `sklearn`. Los datos fueron estandarizados utilizando `MinMaxScaler`. Los hiperparámetros a optimizar incluyeron:

- **Número de capas ocultas y neuronas por capa.** Con un máximo de 64 neuronas por capa y un máximo de 10 capas ocultas.
- **Función de activación.** (ReLU, Tanh, Logistic, Identity).
- **Tasa de aprendizaje.** en un rango de 0.001 a 0.01.
- **Número máximo de iteraciones.** En este caso se estableció un máximo de 1000 iteraciones para evitar sobreajuste, pero el algoritmo genético podría encontrar una configuración que converja antes.
- **Algoritmo de optimización.** (Adam, SGD, LBFGS).
- **Momentum para SGD.** (en caso de ser seleccionado). En un rango de 0.0 a 1.0.

0.3.2. Resultados MLP

Tras la evolución, se obtuvo una precisión final del 100 % (o un accuracy de 1.0) con la siguiente configuración:

Hiperparámetro	Valor Óptimo
Capas Ocultas	[26, 30, 40, 59, 9, 17, 38, 54, 11]
Función de Activación	[Tanh]
Optimizador	[lbfgs]
Tasa de Aprendizaje	[0.008]
Máximo de Iteraciones	[280]
Momentum (SGD)	[0.81]

Cuadro 1: Arquitectura final del modelo MLP.

0.3.3. Análisis de Resultados (MLP)

El modelo MLP logró una precisión perfecta en el conjunto de prueba, lo que indica que la configuración de hiperparámetros encontrada por el algoritmo genético fue altamente efectiva para este dataset. La función de activación Tanh permitió una mejor captura de las relaciones no lineales entre las características del vino, mientras que el optimizador LBFGS facilitó una convergencia rápida y eficiente. La tasa de aprendizaje relativamente baja (0.008) contribuyó a un entrenamiento estable, evitando oscilaciones en la función de pérdida. El número de iteraciones (280) fue suficiente para alcanzar la convergencia sin sobreajustar el modelo, y el momentum de 0.81 para SGD (aunque no se utilizó en este caso) sugiere que el algoritmo genético consideró esta opción como una posible configuración, pero finalmente se decantó por LBFGS, que es un optimizador de segundo orden que no requiere momentum.

0.4. Comparación con una configuración distinta

Para emparar, se provó una configuración manual con los siguientes parámetros:

Hiperparámetro	Valor Manual
Capas Ocultas	[31, 62]
Función de Activación	[ReLU]
Optimizador	[Adam]
Tasa de Aprendizaje	[0.85]
Máximo de Iteraciones	[500]
Momentum (SGD)	[1.0]

Cuadro 2: Configuración manual del modelo MLP.

Esta configuración manual resultó en una precisión del 36.11 %, lo que demuestra la importancia de la optimización de hiperparámetros mediante algoritmos genéticos, ya que una configuración no óptima puede llevar a un rendimiento significativamente inferior. En este caso, la tasa de aprendizaje extremadamente alta (0.85) probablemente causó que el modelo no convergiera adecuadamente, mientras que la función de activación ReLU pudo haber contribuido a problemas de gradientes muertos, especialmente con una arquitectura tan simple como [31, 62]. En contraste, la configuración optimizada por el algoritmo genético permitió una convergencia efectiva y una precisión perfecta, destacando la eficacia de este enfoque para la selección de hiperparámetros en redes neuronales.

En resumen, la optimización de hiperparámetros mediante algoritmos genéticos demostró ser crucial para alcanzar un rendimiento óptimo en el MLP, mientras que una configuración manual sin una búsqueda adecuada puede resultar en un rendimiento significativamente inferior.

0.5. Parte B: Mapa Autoorganizado (SOM)

0.5.1. Implementación Base

Se implementó un SOM de la biblioteca `minisom`. A diferencia del MLP, el SOM no utilizó las etiquetas durante el entrenamiento, cumpliendo con el paradigma de aprendizaje no supervisado. Los datos fueron estandarizados utilizando `MinMaxScaler` para asegurar que todas las características tuvieran la misma escala, lo cual es crucial para el rendimiento del SOM.

Los hiperparámetros a optimizar incluyeron:

- **Dimensiones del mapa.** Se permitió que el algoritmo genético explorara mapas desde 5x5 hasta 30x30.
- **Radio de vecindad (σ).** En un rango de 0.1 a 5.0, para controlar la influencia de las neuronas vecinas durante el entrenamiento.

- **Tasa de aprendizaje.** En un rango de 0.01 a 0.1, para ajustar la velocidad de actualización de los pesos.
- **Función de vecindad.** Se consideraron funciones como gaussiana, triangular, mexican hat y bubble para evaluar cuál se adapta mejor a la estructura del dataset.
- **Número de iteraciones.** Se estableció un máximo de 10000 iteraciones para permitir una convergencia adecuada sin sobreentrenar el modelo.

0.5.2. Resultados de Optimización SOM

Después de la evolución del algoritmo genético, la mejor solución encontrada arrojó un **Fitness (ARI) de 0.967**. Los parámetros óptimos decodificados fueron:

- **Dimensiones del mapa:** 9 de ancho por 24 de alto.
- **Radio de vecindad (σ):** 2.07.
- **Tasa de aprendizaje:** 0.09.
- **Función de vecindad:** gaussian.
- **Iteraciones:** 1278.

0.5.3. Análisis de Resultados (SOM)

El valor de ARI de 0.967 indica una separación casi perfecta de las tres clases de vino. La topología del mapa (9x24) sugiere que una estructura alargada permitió una mejor discriminación de las características lineales del dataset. La función de vecindad **gaussian** demostró ser más efectiva que la triangular, mexican hat y bubble para este conjunto, permitiendo una adaptación más nítida de las neuronas a la densidad de los datos.

0.6. Análisis de Resultados (SOM)

El valor de ARI de 0.967 indica una separación casi perfecta de las tres clases de vino. La topología del mapa (9x24) sugiere que una estructura alargada permitió una mejor discriminación de las características lineales del dataset. La función de vecindad **gaussian** demostró ser más efectiva que la triangular, mexican hat y bubble para este conjunto, permitiendo una adaptación más nítida de las neuronas a la densidad de los datos. La tasa de aprendizaje relativamente alta (0.09) facilitó una convergencia rápida sin caer en mínimos locales, mientras que el radio de vecindad de 2.07 permitió una influencia adecuada entre las neuronas durante el entrenamiento, optimizando la competencia y cooperación entre ellas. En resumen, la optimización de hiperparámetros mediante algoritmos genéticos fue crucial para alcanzar un rendimiento óptimo en el SOM, demostrando que la configuración adecuada de parámetros puede llevar a una excelente correspondencia entre los clusters generados y las clases reales del

dataset. En contraste, una configuración manual sin una búsqueda adecuada podría resultar en un rendimiento significativamente inferior, subrayando la importancia de la optimización automatizada en modelos de aprendizaje no supervisado.

0.7. Comparación con otra configuración

Para comparar, se probó una configuración manual con los siguientes parámetros:

- **Dimensiones del mapa:** 7 de ancho por 7 de alto.
- **Radio de vecindad (σ):** 1.
- **Tasa de aprendizaje:** 0.01.
- **Función de vecindad:** `triangle`.
- **Iteraciones:** 1000.

Esta configuración manual resultó en un ARI de 0.45, lo que demuestra la importancia de la optimización de hiperparámetros mediante algoritmos genéticos. La topología del mapa (7x7) probablemente no fue suficiente para capturar la complejidad del dataset, mientras que la función de vecindad `triangular` pudo haber contribuido a una adaptación menos precisa de las neuronas a la densidad de los datos. La tasa de aprendizaje baja (0.01) también pudo haber ralentizado la convergencia, resultando en un modelo subóptimo. En contraste, la configuración optimizada por el algoritmo genético permitió una convergencia efectiva y un ARI cercano a 1, destacando la eficacia de este enfoque para la selección de hiperparámetros en modelos de aprendizaje no supervisado.

0.8. Conclusión

El uso de algoritmos genéticos eliminó la necesidad de realizar un ajuste manual de prueba y error. Para el MLP, se logró una convergencia rápida gracias a la optimización de la tasa de aprendizaje. Para el SOM, la alta sensibilidad a los parámetros de vecindad y dimensiones fue manejada exitosamente por PyGAD, demostrando que la competencia y cooperación entre neuronas se optimiza mejor cuando el radio inicial está alineado con la densidad de los datos escalados.

0.9. Ejecución

El algoritmo Genético para MLP tiene por nombre: `Examen2_parteA.py` y el de SOM es `Examen2_parteB.py`. Ambos scripts pueden ser ejecutados directamente en un entorno Python con las bibliotecas necesarias instaladas. Para probar los resultados obtenidos se utilizan algoritmos en los archivos: `probar_best_solution.py` y `mini.py`, los cuales contienen las configuraciones manuales para comparar con los resultados optimizados por el algoritmo genético.